

Feature Scaling & Encoding – Cell-by-Cell Explanation

ML Developer Explanation of the Uploaded Jupyter Notebook

Source: Feature Scaling, Encoding.ipynb

This PDF follows the uploaded notebook's original 45-cell order. The explanations distinguish what the notebook actually does from developer notes where appropriate.

Item	Notebook result
Cells	45
Dataset	Iris
Rows after duplicate removal	149
Input features	4 numerical features
Target	species
Split	119 train / 30 test
Preprocessing	Median imputation + StandardScaler
Model	Logistic Regression
Training accuracy	98.32%
Testing accuracy	93.33%
Macro F1	92.80%

1. Complete ML Workflow in This Notebook

Stage	What happens
1. Data gathering	Load Iris CSV into pandas.
2. Quality checks	Shape, info, duplicates, missing values.
3. X/Y separation	Four measurements → X; species → Y.
4. Train/test split	80/20 split with random_state=21.
5. Preprocessing	Median imputation + StandardScaler.
6. Model	Logistic Regression.
7. Evaluation	Accuracy, macro F1, confusion matrix, classification report.
8. Class balance	Count and plot species distribution.
9. Save artifacts	Save model and preprocessing pipeline with joblib.
10. Inference	Load new data → preprocess → predict → export CSV.

Important notebook-specific point: The file is titled Feature Scaling, Encoding, but the actual cells do not use `OneHotEncoder` or another explicit feature-encoding transformer. The input features are already numerical, while the target `species` remains text labels for Logistic Regression.

Cell 1 – Notebook objective and setup

Cell type: markdown

Original notebook content

```
## Feature Engineering: Feature Scaling(Numerical data) and Feature Encoding(Categorical data)
```

Business Objective: Build a model that can predict species of Iris flower on basis of flower petal length/width and sepal length/width

```
## Step1: Data Gathering
```

What this cell does

The notebook introduces Feature Engineering with two ideas: Feature Scaling for numerical data and Feature Encoding for categorical data. Its stated business objective is to predict the species of an Iris flower from sepal length/width and petal length/width.

ML Developer understanding

The actual modeling workflow in this notebook uses numerical feature scaling and Logistic Regression. Although the title mentions encoding, the notebook does not contain a OneHotEncoder step for the target or input features.

Cell 2 – Load the Iris dataset

Cell type: code

Original notebook content

```
import pandas as pd
path=r"https://raw.githubusercontent.com/sindhura-nk/Datasets/refs/heads/main/iris.csv"
df = pd.read_csv(path)
df.head()
```

What this cell does

`pandas` is imported and `pd.read\_csv()` loads the Iris CSV from the supplied source URL. `df.head()` displays the first five rows.

ML Developer understanding

The dataset has four numerical measurements and one categorical target column, `species`.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Cell 3 – Data-quality section

Cell type: markdown

Original notebook content

```
### Step2: Perform basic data quality checks
```

What this cell does

This markdown cell marks the beginning of the basic data-quality checks.

ML Developer understanding

Before training a model, inspect size, types, duplicates, and missing values.

Cell 4 – Check shape

Cell type: code

Original notebook content

```
df.shape
```

What this cell does

`df.shape` returns `(150, 5)`, meaning 150 records and 5 columns.

ML Developer understanding

There are four predictor columns and one target column.

Stored notebook output

```
(150, 5)
```

Cell 5 – Inspect DataFrame information

Cell type: code

Original notebook content

```
df.info()
```

What this cell does

`df.info()` shows 150 non-null values in every column. The four measurements are `float64`; `species` is `object`.

ML Developer understanding

This confirms the predictors are numerical while the target is categorical text.

Stored notebook output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null    float64
1   sepal_width     150 non-null    float64
2   petal_length    150 non-null    float64
3   petal_width     150 non-null    float64
4   species         150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

Cell 6 – Check duplicate rows

Cell type: code

Original notebook content

```
df.duplicated().sum()
```

What this cell does

`df.duplicated().sum()` finds 1 duplicate row.

ML Developer understanding

Duplicate records can give repeated observations extra influence, so the notebook removes them in the next cell.

Stored notebook output

```
np.int64(1)
```

Cell 7 – Remove duplicates

Cell type: code

Original notebook content

```
df=df.drop_duplicates()
df.duplicated().sum()
```

What this cell does

`df.drop_duplicates()` removes the duplicate. The second check returns 0 duplicates.

ML Developer understanding

The working dataset now contains 149 unique rows. This also explains why the later train/test split has $119 + 30 = 149$ records.

Stored notebook output

```
np.int64(0)
```

Cell 8 – Check missing values

Cell type: code

Original notebook content

```
# missing data
df.isna().sum()
```

What this cell does

`df.isna().sum()` returns zero for every column.

ML Developer understanding

The original Iris training dataset has no missing values, but the notebook still keeps an imputer in its preprocessing pipeline. That becomes useful later because the out-of-sample file contains a missing petal-length value.

Stored notebook output

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

Cell 9 – Separate X and Y

Cell type: markdown

Original notebook content

```
## Separate X and Y features
Predict Species: Y
```

What this cell does

This markdown cell introduces the supervised-learning split: X contains predictors and Y contains the species target.

ML Developer understanding

For classification, X is the input matrix and y/Y is the class label.

Cell 10 – Create X and Y

Cell type: code

Original notebook content

```
X = df.drop(columns='species')
Y = df[['species']]
print("X records")
display(X.head())
print("Y records")
display(Y.head())
```

What this cell does

`X = df.drop(columns='species')` keeps the four numerical predictors. `Y = df[['species']]` keeps the target as a one-column DataFrame. The notebook displays the first rows of both.

ML Developer understanding

This creates the standard supervised-learning structure: $X \rightarrow \text{model} \rightarrow \text{predicted species}$.

Stored notebook output

X records

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Y records

	species
0	setosa
1	setosa
2	setosa
3	setosa
4	setosa

Cell 11 – Train/test split section

Cell type: markdown

Original notebook content

```
### Step4: SPlit the data into training and testing
```

```
    training: 80% and testing: 20%
```

What this cell does

The notebook specifies 80% training data and 20% testing data.

ML Developer understanding

The test set should remain unseen during model fitting so it can estimate generalization.

Cell 12 – Split the dataset

Cell type: code

Original notebook content

```
from sklearn.model_selection import train_test_split

xtrain,xtest,ytrain,ytest = train_test_split(X,Y,train_size=0.8,random_state=21)
print(xtrain.shape,xtest.shape)
print(ytrain.shape,ytest.shape)
```

What this cell does

`train\_test\_split()` uses `train\_size=0.8` and `random\_state=21`. Because there are 149 unique records, the split produces 119 training rows and 30 test rows.

ML Developer understanding

The fixed random state makes the same split reproducible.

Stored notebook output

```
(119, 4) (30, 4)
(119, 1) (30, 1)
```

Cell 13 – Data processing section

Cell type: markdown

Original notebook content

```
### Data Processing: Data Cleaning
```

What this cell does

This markdown cell marks the data-cleaning/preprocessing stage.

ML Developer understanding

The notebook prepares numerical features before Logistic Regression.

Cell 14 – Check feature names

Cell type: code

Original notebook content

```
xtrain.columns
```

What this cell does

`xtrain.columns` confirms the four numerical feature names: sepal_length, sepal_width, petal_length, and petal_width.`

ML Developer understanding

Knowing the feature names is useful when building and validating preprocessing.

Stored notebook output

```
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width'], dtype='object')
```

Cell 15 – Check feature data types

Cell type: code

Original notebook content

```
xtrain.dtypes
```

What this cell does

`xtrain.dtypes` shows all four predictors are `float64`.`

ML Developer understanding

All input variables are numerical, so a numerical preprocessing pipeline is appropriate.

Stored notebook output

```
sepal_length    float64
sepal_width     float64
petal_length     float64
petal_width     float64
dtype: object
```

Cell 16 – Create preprocessing pipeline

Cell type: code

Original notebook content

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

num_pipe = make_pipeline(
    SimpleImputer(strategy='median'), # handling missing data
    StandardScaler() # scale all the numerical values
).set_output(transform='pandas')
```

What this cell does

The pipeline contains `SimpleImputer(strategy='median')` followed by StandardScaler()`. set_output(transform='pandas')` requests pandas DataFrame output.`

ML Developer understanding

Median imputation fills missing numerical values using the training median. StandardScaler standardizes features by removing the mean and scaling to unit variance. This is a common preprocessing step for algorithms such as Logistic

Cell 17 – Fit the preprocessing pipeline

Cell type: code

Original notebook content

```
# fit it on training data
num_pipe.fit(xtrain)
```

What this cell does

`num\_pipe.fit(xtrain)` learns the preprocessing parameters from training data only. The stored pipeline contains the imputer and scaler.

ML Developer understanding

Fitting only on training data helps avoid test-data leakage.

Stored notebook output

```
Pipeline(steps=[('simpleimputer', SimpleImputer(strategy='median')),
                 ('standardscaler', StandardScaler())])
```

Cell 18 – Transform train and test data

Cell type: code

Original notebook content

```
# transform it on testing as well as training data
xtrain_pre = num_pipe.transform(xtrain)
xtest_pre = num_pipe.transform(xtest)
```

What this cell does

The fitted pipeline transforms both `xtrain` and `xtest`. The same learned statistics are used for both sets.

ML Developer understanding

This keeps preprocessing consistent between training and testing.

Cell 19 – Inspect transformed training data

Cell type: code

Original notebook content

```
xtrain_pre.head()
```

What this cell does

`xtrain\_pre.head()` shows standardized feature values such as -1.58 and 1.78 rather than the original centimeter measurements.

ML Developer understanding

After standardization, features are on comparable scales, which can help distance- and optimization-based algorithms.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width
41	-1.583790	-1.803633	-1.400001	-1.186568
130	1.785913	-0.626921	1.275411	0.867273
70	0.042963	0.314449	0.550820	0.738908
46	-0.886610	1.726504	-1.232788	-1.314933
125	1.553520	0.314449	1.219673	0.738908

Cell 20 – Inspect transformed test data

Cell type: code

Original notebook content

```
xtest_pre.head()
```

What this cell does

`xtest\_pre.head()` shows the test features after the same preprocessing.

ML Developer understanding

The test data is transformed with the training-fitted pipeline rather than fitting a new scaler.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width
92	-0.073233	-1.097606	0.104918	-0.031282
44	-0.886610	1.726504	-1.065575	-1.058203
7	-1.002806	0.785134	-1.288526	-1.314933
21	-0.886610	1.491161	-1.288526	-1.058203
95	-0.189430	-0.156236	0.216394	-0.031282

Cell 21 – Model-building section

Cell type: markdown

Original notebook content

```
### Model Building: Logistic Regression
```

What this cell does

This markdown cell introduces Logistic Regression.

ML Developer understanding

Despite its name, Logistic Regression is a classification algorithm. Here it predicts one of the three Iris species.

Cell 22 – Import Logistic Regression

Cell type: code

Original notebook content

```
from sklearn.linear_model import LogisticRegression
```

What this cell does

`LogisticRegression` is imported from scikit-learn.

ML Developer understanding

This estimator is used for the multiclass classification task.

Cell 23 – Create and train the model

Cell type: code

Original notebook content

```
model = LogisticRegression()  
model.fit(xtrain_pre, ytrain)
```

What this cell does

`LogisticRegression()` creates the classifier and `model.fit(xtrain\_pre, ytrain)` trains it. The notebook records a `DataConversionWarning` because Y is a one-column DataFrame; scikit-learn expects a 1D target for this estimator.

ML Developer understanding

The warning does not stop training, but a cleaner implementation would pass `ytrain.values.ravel()` or use a Series. This is a developer note, not a change to the original notebook.

Stored notebook output

```
e:\DATA SCIENTIST\ML\repository\venv\lib\site-packages\sklearn\utils\validation.py:1406: DataConversionWarning: A column-vector y was passed as a 1D array, but it should be a 2D array if the label is not a scalar
  y = column_or_1d(y, warn=True)
```

```
LogisticRegression()
```

Cell 24 – Training accuracy

Cell type: code

Original notebook content

```
# training scores(accuracy)
model.score(xtrain_pre,ytrain)
```

What this cell does

`model.score(xtrain_pre, ytrain)` returns approximately `0.9832`, or 98.32% training accuracy.

ML Developer understanding

The model correctly classifies almost all training samples.

Stored notebook output

```
0.9831932773109243
```

Cell 25 – Testing accuracy

Cell type: code

Original notebook content

```
# testing scores(accuracy)
model.score(xtest_pre,ytest)
```

What this cell does

`model.score(xtest_pre, ytest)` returns `0.9333`, or 93.33% test accuracy.

ML Developer understanding

The model performs well on unseen test data, although the test set contains only 30 observations.

Stored notebook output

```
0.9333333333333333
```

Cell 26 – Performance-metrics section

Cell type: markdown

Original notebook content

```
## Training score as well testing scores are above 80%. We can consider this model. But for further evaluation, let's check for performance metrics.

precision
recall
f1_score
accuracy
```

What this cell does

The notebook says both training and testing scores are above 80% and decides the model can be considered, then requests precision, recall, F1-score, and accuracy for further evaluation.

ML Developer understanding

Accuracy alone can hide class-specific errors, so precision, recall, F1, and the confusion matrix provide more detail.

Cell 27 – Generate predictions

Cell type: code

Original notebook content

```
ytest_preds = model.predict(xtest_pre)
ytest_preds[:5]
```

What this cell does

`model.predict(xtest\_pre)` generates a predicted species for each test observation. The first five predictions are versicolor, setosa, setosa, setosa, and versicolor.

ML Developer understanding

These are the classifier's final class predictions, not probabilities.

Stored notebook output

```
array(['versicolor', 'setosa', 'setosa', 'setosa', 'versicolor'],
      dtype=object)
```

Cell 28 – Inspect actual test labels

Cell type: code

Original notebook content

```
ytest.head()
```

What this cell does

`ytest.head()` shows the first five true labels corresponding to the same test observations.

ML Developer understanding

Comparing true labels with predictions is the basis for classification metrics.

Stored notebook output

```
species
92  versicolor
44   setosa
7    setosa
21   setosa
95  versicolor
```

Cell 29 – Calculate macro F1-score

Cell type: code

Original notebook content

```
from sklearn.metrics import f1_score
f1_score_data = f1_score(ytest, ytest_preds, average='macro')
f1_score_data
```

What this cell does

`f1\_score(..., average='macro')` calculates the F1-score independently for each class and then gives equal weight to the classes. The result is about `0.9280`.

ML Developer understanding

Macro F1 is useful when you want each class to contribute equally to the overall F1 result.

Stored notebook output

```
0.9280303030303031
```

Cell 30 – Confusion matrix section

Cell type: markdown

Original notebook content

```
### Observe Confusion matrix and Classification Report
```

What this cell does

This markdown cell introduces the confusion matrix and classification report.

ML Developer understanding

These tools show where the classifier is correct and where it confuses one species with another.

Cell 31 – Display confusion matrix

Cell type: code

Original notebook content

```
from sklearn.metrics import ConfusionMatrixDisplay, classification_report
ConfusionMatrixDisplay.from_estimator(model, xtest_pre, ytest)
```

What this cell does

`ConfusionMatrixDisplay.from\_estimator()` creates a confusion matrix from the fitted model and test data.

ML Developer understanding

Rows/columns show how true classes and predicted classes relate; the diagonal represents correct predictions.

Stored notebook output

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x27fb608fc40>
<Figure size 640x480 with 2 Axes>

Cell 32 – Classification report

Cell type: code

Original notebook content

```
print(classification_report(ytest, ytest_preds))
```

What this cell does

`classification\_report()` reports precision, recall, F1-score, and support for each species. The stored result shows overall accuracy of 0.93 on 30 test samples.

ML Developer understanding

The report shows setosa with 1.00 precision, 1.00 recall, 1.00 F1; versicolor with 1.00 precision, 0.83 recall, 0.91 F1; and virginica with 0.78 precision, 1.00 recall, 0.88 F1.

Stored notebook output

precision	recall	f1-score	support		
	setosa	1.00	1.00	1.00	11
	versicolor	1.00	0.83	0.91	12
	virginica	0.78	1.00	0.88	7
	accuracy			0.93	30
	macro avg	0.93	0.94	0.93	30
	weighted avg	0.95	0.93	0.93	30

Cell 33 – Class-balance section

Cell type: markdown

Original notebook content

```
## Imbalanced Data: Classification
```

```
Target feature
```

```
Datapoints:
```

```
Class A
```

```
Class B
```

```
df['species'].value_counts()
```

What this cell does

The markdown cell introduces checking class distribution under the heading 'Imbalanced Data: Classification'.

ML Developer understanding

Class balance matters because severe imbalance can make accuracy misleading. The actual Iris data here is nearly balanced.

Cell 34 – Count species classes

Cell type: code

Original notebook content

```
df['species'].value_counts()
```

What this cell does

`value_counts()` shows setosa = 50, versicolor = 50, and virginica = 49 after duplicate removal.

ML Developer understanding

The classes are nearly balanced, so there is no strong class-imbalance problem in this dataset.

Stored notebook output

```
species
setosa      50
versicolor  50
virginica    49
Name: count, dtype: int64
```

Cell 35 – Plot class counts

Cell type: code

Original notebook content

```
df['species'].value_counts().plot(kind='bar')
```

What this cell does

The value counts are plotted as a bar chart.

ML Developer understanding

A bar chart gives a quick visual check of class frequency.

Stored notebook output

```
<Axes: xlabel='species'>
<Figure size 640x480 with 1 Axes>
```

Cell 36 – Save model and pipeline

Cell type: markdown

Original notebook content

```
## Save the model as well as pipeline
```

What this cell does

This markdown cell introduces model persistence.

ML Developer understanding

Saving both the trained model and preprocessing pipeline is important because future input must go through the same preprocessing before prediction.

Cell 37 – Save artifacts with joblib

Cell type: code

Original notebook content

```
import joblib
joblib.dump(model, "IrisPredictionModel.joblib")
joblib.dump(num_pipe, "pipeline_cleaning.joblib")
```

What this cell does

`joblib.dump()` saves the Logistic Regression model as `IrisPredictionModel.joblib` and the preprocessing pipeline as `pipeline_cleaning.joblib`.

ML Developer understanding

The two files together form the reusable prediction system: preprocess new input with the pipeline, then send it to the trained model.

Stored notebook output

```
['pipeline_cleaning.joblib']
```

Cell 38 – Out-of-sample prediction section

Cell type: markdown

Original notebook content

```
### Picking up sample data for out of sample predictions
```

What this cell does

This markdown cell introduces prediction on new data that was not part of the training/test split.

ML Developer understanding

This simulates a real-world inference workflow.

Cell 39 – Load new sample data

Cell type: code

Original notebook content

```
xnew = pd.read_csv(r"https://raw.githubusercontent.com/sindhura-nk/Datasets/refs/heads/main/iris_sample.csv")
xnew.head()
```

What this cell does

`pd.read_csv()` loads `iris_sample.csv`. The displayed sample includes four numerical features; row 4 contains a missing `petal_length` value.

ML Developer understanding

This is a useful demonstration of why the saved imputer is included in the pipeline.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width
0	5.5	2.5	4.0	1.3
1	6.9	3.1	5.1	2.3
2	5.1	2.5	3.0	1.1
3	4.4	2.9	NaN	0.2
4	5.9	3.0	5.1	1.8

Cell 40 – Load pipeline and preprocess new data

Cell type: code

Original notebook content

```
pipeline_loaded = joblib.load("pipeline_cleaning.joblib")
```

```
xnew_pre = pipeline_loaded.transform(xnew)
xnew_pre.head()
```

What this cell does

The saved pipeline is loaded with `joblib.load()`, then `transform(xnew)` applies the same median-imputation and standardization steps used during training. The missing value is handled by the imputer.

ML Developer understanding

Using the saved preprocessing pipeline prevents training-time and prediction-time preprocessing from becoming inconsistent.

Stored notebook output

```
sepal_length  sepal_width  petal_length  petal_width
0      -0.421823    -1.332948     0.104918     0.097083
1       1.204930     0.079107     0.718033     1.380733
2      -0.886610    -1.332948    -0.452459    -0.159647
3      -1.699986    -0.391578     0.383607    -1.314933
4       0.042963    -0.156236     0.718033     0.738908
```

Cell 41 – Load model and predict species

Cell type: code

Original notebook content

```
model_loaded = joblib.load("IrisPredictionModel.joblib")
Species_Prediction = model_loaded.predict(xnew_pre)
Species_Prediction[:5]
```

What this cell does

The saved model is loaded and used to predict the new samples. The first five predictions are versicolor, virginica, versicolor, setosa, and virginica.

ML Developer understanding

This is the final inference step of the notebook.

Stored notebook output

```
array(['versicolor', 'virginica', 'versicolor', 'setosa', 'virginica'],
      dtype=object)
```

Cell 42 – Review original new samples

Cell type: code

Original notebook content

```
xnew.head()
```

What this cell does

`xnew.head()` displays the original input measurements again, including the missing value in row 3.

ML Developer understanding

This lets you compare raw inputs with the predictions added in the next cell.

Stored notebook output

```
sepal_length  sepal_width  petal_length  petal_width
0           5.5          2.5           4.0          1.3
1           6.9          3.1           5.1          2.3
2           5.1          2.5           3.0          1.1
3           4.4          2.9           NaN          0.2
4           5.9          3.0           5.1          1.8
```

Cell 43 – Attach predictions

Cell type: code

Original notebook content

```
xnew["Predicted_Species"] = Species_Prediction
xnew.head()
```

What this cell does

`Predicted\_Species` is added as a new column. The displayed output shows the five predictions alongside the original measurements.

ML Developer understanding

This creates a user-friendly prediction result table.

Stored notebook output

	sepal_length	sepal_width	petal_length	petal_width	Predicted_Species
0	5.5	2.5	4.0	1.3	versicolor
1	6.9	3.1	5.1	2.3	virginica
2	5.1	2.5	3.0	1.1	versicolor
3	4.4	2.9	NaN	0.2	setosa
4	5.9	3.0	5.1	1.8	virginica

Cell 44 – Export predictions

Cell type: code

Original notebook content

```
xnew.to_csv("IrisSpeciesPredictedResults.csv", index=False)
```

What this cell does

`xnew.to\_csv(..., index=False)` saves the prediction results to `IrisSpeciesPredictedResults.csv` without writing the pandas index as a separate column.

ML Developer understanding

This produces a shareable CSV output for downstream use.

Cell 45 – Empty cell

Cell type: code

What this cell does

This cell contains no code or output.

ML Developer understanding

No additional processing occurs.

2. Key Concepts You Should Understand

Concept	Simple explanation
Feature Scaling	Changes numerical features to a comparable scale so algorithms are not unnecessarily dominated by large numerical values.
StandardScaler	Centers each feature around mean 0 and scales it to unit variance. scikit-learn documentation
Median Imputation	Replaces missing numerical values with the median learned from training data.
Pipeline	Chains preprocessing steps so the same fitted transformations can be reused consistently.
Logistic Regression	A classification algorithm that learns class decision boundaries from input features.
Accuracy	Correct predictions divided by total predictions.
Precision	Among predictions for a class, the fraction that are actually that class.
Recall	Among actual samples of a class, the fraction correctly found.
F1-score	Harmonic mean of precision and recall.
Macro F1	Calculates F1 per class and gives each class equal weight.
Confusion Matrix	Shows true-versus-predicted class counts.
One-Hot Encoding	Converts categories into binary indicator columns. The notebook title mentions encoding, but no OneHotEncoder class is used.

Why scaling matters: scikit-learn describes feature scaling as an important preprocessing step for many algorithms. Standardization commonly produces mean 0 and standard deviation 1; tree-based models are much less affected by scaling than many other algorithms. [scikit-learn documentation](#)

3. Results and ML Developer Observations

Result	Value	Meaning
Unique rows	149	One duplicate was removed from the original 150 rows.
Training accuracy	98.32%	Very strong performance on training data.
Test accuracy	93.33%	Strong performance on the 30 unseen test rows.
Macro F1	92.80%	Balanced class-level F1 summary.
Setosa	P=1.00, R=1.00, F1=1.00	Perfect on the stored test report.
Versicolor	P=1.00, R=0.83, F1=0.91	Some actual versicolor samples were missed.
Virginica	P=0.78, R=1.00, F1=0.88	All actual virginica were found, but some predictions were false positives.
Class counts	50 / 50 / 49	Nearly balanced; no severe class imbalance.

Developer note on Cell 23: The notebook produced a `DataConversionWarning` because ``ytrain`` is a one-column `DataFrame`. For cleaner scikit-learn usage, a 1D target such as ``ytrain.squeeze()`` or ``ytrain.values.ravel()`` is typically preferable. The original notebook is preserved exactly in the cell-by-cell section.

Developer note on the title: The notebook is called Feature Scaling, Encoding, but the actual workflow performs scaling and missing-value imputation; it does not explicitly encode the numerical input features or the string target with `OneHotEncoder`.

4. Interview-Ready Answers

What is the purpose of feature scaling?

Feature scaling puts numerical features on comparable scales. In this notebook, StandardScaler is used before Logistic Regression.

Why did you use StandardScaler?

Because the four Iris predictors can be placed on a standardized scale with mean 0 and unit variance, making the numerical inputs comparable.

Why did you use a pipeline?

The pipeline combines median imputation and StandardScaler so the exact same preprocessing can be fitted on training data and reused for test and future data.

Why did you remove duplicates?

The dataset initially contained one duplicate row. The notebook removed it so the same observation would not appear twice.

What was the model accuracy?

Training accuracy was about 98.32% and testing accuracy was about 93.33%.

Why use precision, recall and F1 in addition to accuracy?

They show class-specific performance and can reveal errors that a single accuracy value can hide.

What is the macro F1-score here?

Approximately 0.928, or 92.80%. It gives equal importance to each species class.

How does the saved pipeline help prediction?

The new input is passed through the saved imputer and scaler before being passed to the saved Logistic Regression model, keeping preprocessing consistent.

What happened with missing data in the new sample?

The out-of-sample data contains a missing petal_length value. The saved median-imputation step handles that missing value before prediction.

5. Final End-to-End Understanding

Raw Iris data → quality checks → remove duplicate → X/Y split → 80/20 train/test split → median imputation + StandardScaler → Logistic Regression → predictions → accuracy/F1/confusion matrix/classification report → class-balance check → save model + pipeline → load new sample → preprocess with saved pipeline → predict species → export CSV.